SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing Department of Software Engineering

SE3090 Student Model Examination 2026

Software Engineering Frameworks Year 3 Semester 1

Duration 2 hours Total marks 100 Answer all 30 questions

Practice the examination structure and answer formats using the questions below. Attempt the paper before consulting the separate marking scheme. This is a student practice paper.

Instructions to candidates

1. Answer ALL questions in six sections A to F. Marks are shown for every question and subpart.

2. Section A has 15 MCQs worth 1 mark each. Select exactly one option for single-answer items. Items labelled Select ALL that apply have more than one correct option.

3. For this model, a single-answer MCQ earns 1 only for the correct single selection. A multiple-answer item awards an equal share for each correct selection and deducts that share for each wrong selection, with a minimum of zero for that item.

4. For numbered code blanks, write [N] followed by the missing code only. Blank numbering restarts in each question. Do not copy the full program unless requested.

5. For explanations and decisions, answer the stated points and connect your reasons to the scenario. Correct equivalent implementations can earn marks.

6. Use C# / ASP.NET Core, JavaScript / React and Dart / Flutter as requested. Code may be typed as plain text; exact indentation is not assessed.

7. Use separate answer boxes or clearly numbered responses, following the NetExam style. Attempt under timed closed-book conditions, then self-mark using the companion scheme.

Paper structure and suggested time

Section	Questions	Marks	Minutes
A	1-15	15	15
B	16-19	30	35
C	20-22	15	17
D	23-25	15	17
E	26-27	10	11
F	28-30	15	15
Total	1-30	100	110

Reserve the remaining 10 minutes for reading and checking your answers. The suggested times are a practice guide.

Section A Multiple choice questions

Questions 1 to 15 15 marks Each question carries 1 mark

Question 1 1 mark

A campus app needs to resize a photograph whenever a new image is uploaded. The team does not want to administer a virtual machine for this small event-driven task. Which approach best matches this requirement?

a. A long-running desktop application on a staff computer
b. A serverless function triggered by the upload event
c. A relational database index
d. A React component that runs only in the browser
e. A Git branch for each uploaded image

Question 2 1 mark

Before choosing a framework for a three-week prototype, which of the following is/are useful evidence for the decision? Select ALL that apply.

a. The team can already build and debug a small feature with it
b. Its logo looks more professional than the alternatives
c. A social media poll says it is the newest framework
d. A small prototype meets the project requirements under the expected workload
e. Its licensing and integration requirements fit the project constraints

Question 3 1 mark

A React page performs an expensive calculation to group tickets by priority. The calculation depends on the tickets array and the selected room. Which hook can cache the calculated value between renders until those dependencies change?

a. useRef
b. useEffect
c. useCallback
d. useMemo
e. useContext

Question 4 1 mark

Consider C# code: var roomId = 12; followed by roomId = "R12";. Which statement is correct?

a. Both statements compile because var allows the variable to change type
b. roomId becomes an object after the second statement
c. The runtime silently converts "R12" to an integer
d. var makes roomId a dynamically typed variable
e. The second statement fails to compile because roomId was inferred as int

Question 5 1 mark

The team adds a Priority property to its EF Core Ticket entity. What is the main purpose of creating a migration for this change?

a. To issue a JWT for every existing ticket
b. To compile the React production bundle
c. To record a versioned database schema change that can be applied to the database
d. To replace all controller routes automatically
e. To remove the need for database constraints

Section A continued

Question 6 1 mark

A ticket list repeatedly loads each ticket's related Room in separate database queries. Which of the following is/are sensible responses to this potential N+1 query problem? Select ALL that apply.

a. Assume that using an ORM guarantees efficient queries
b. Consider eager loading the related Room with Include when that data is needed
c. Add more HTTP redirects to the list endpoint
d. Inspect the generated SQL and query count to verify the problem and the improvement
e. Remove all foreign keys from the database

Question 7 1 mark

A developer has made a local commit that has not been pushed. They want to remove that last commit while keeping its changes staged for a revised commit. Which command matches that intention?

a. git reset --soft HEAD~1
b. git reset --hard HEAD~1
c. git restore .
d. git push --force
e. git branch -D main

Question 8 1 mark

In a Flutter Row, a ticket-title TextField should occupy the remaining width beside a fixed-size send icon. Which widget should wrap the TextField?

a. Scaffold
b. Navigator
c. Center
d. Expanded
e. AppBar

Question 9 1 mark

In Dart, String? note = null; is followed by final shown = note ?? "No note";. What value is assigned to shown?

a. null
b. "No note"
c. An empty string
d. The literal text "note"
e. A compile-time error in all cases

Question 10 1 mark

A browser loads the dashboard from https://portal.example.edu and calls an API at https://api.example.edu. Both addresses are fictitious. The API must permit that browser origin. Which configuration is relevant?

a. An additional Git remote
b. A Dart import statement
c. An API CORS policy that allows the dashboard origin
d. A database composite primary key
e. A Flutter theme

Section A continued

Question 11 1 mark

Which of the following is/are consistent with the React hook rules and side-effect practices taught in the module? Select ALL that apply.

a. Call hooks at the top level of a function component or custom hook
b. Call useState only after a condition becomes true
c. Clean up timers or subscriptions created by an effect
d. Call a hook inside each iteration of a map loop
e. Use a dependency list that reflects the values used by the effect

Question 12 1 mark

Which of the following is/are valid matches between a PostgreSQL constraint and its purpose? Select ALL that apply.

a. CHECK (priority BETWEEN 1 AND 5) prevents two users from sharing an email address
b. UNIQUE (room_code) prevents duplicate room codes
c. A foreign key guarantees that every API request has a valid JWT
d. NOT NULL on title prevents a database NULL title value
e. A foreign key from tickets.room_id to rooms.id prevents a ticket from referencing a nonexistent room

Question 13 1 mark

Which of the following is/are correct matches for automated security checks? Select ALL that apply.

a. A code scanner such as CodeQL analyses source code for vulnerable patterns
b. A formatter proves that the authentication logic is secure
c. A dependency audit checks third-party packages for known vulnerabilities
d. An application screenshot detects a leaked signing key in Git history
e. Secret scanning can detect exposed credentials in repository content

Question 14 1 mark

Which Flutter test type is the best fit for checking, in isolation, that tapping a screen's Submit button displays a validation error beside an empty field?

a. A widget test
b. A test of the cloud provider's billing account
c. Only a full production integration test
d. A Git merge test
e. A SQL schema migration

Question 15 1 mark

Which of the following is/are useful observations about an agent execution trace? Select ALL that apply.

a. It can record tool names, arguments, results and errors across a run
b. It guarantees that every answer is factually correct
c. It is identical to a single final-answer string
d. It can reveal repeated tool calls and where time or cost was spent
e. It removes the need to evaluate the assistant on known test cases

Section B Backend development with ASP NET Core and PostgreSQL

Questions 16 to 19 30 marks

CampusCare is a university maintenance portal. Reporters submit faults for rooms; technicians investigate and an authorized dispatcher assigns work. The backend uses C# / ASP.NET Core, PostgreSQL and EF Core with Npgsql. New tickets have a title, room and priority from 1 to 5; the server assigns their id and initial Open status. The public reporting endpoint is a demonstration simplification for this model paper. The assignment endpoint is protected separately in Question 19.

Question 16 API contracts and testable design (6 marks)

16.1 Ticket has Id, Title, RoomId, Priority, Status and InternalStaffNote properties. Design the input contract for creating a report: list the THREE fields that a reporter should supply and explain why Id and InternalStaffNote should be excluded. Status is assigned by the server. **(2 marks)**

16.2 TicketsController receives an ITicketService through its constructor. Explain how this helps you test the controller without a real database. State what you would inject and one benefit for the test. **(2 marks)**

16.3 The database query returns a Task<List<TicketSummaryDto>>. Explain what await does while the database operation is incomplete and why this is preferable to blocking on .Result in a web request. **(2 marks)**

Question 17 Database constraints and schema changes (6 marks)

17.1 Complete only blanks [1]-[4], one numbered answer per blank. The schema must reject a missing title, reject priorities outside 1-5, reference an existing room and set a new ticket's status to Open by default. Each blank carries 1 mark. **(4 marks)**

```
CREATE TABLE rooms (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  room_code VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE tickets (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  room_id BIGINT NOT NULL __________ rooms(id), -- [1]
  title VARCHAR(120) __________,                -- [2]
  priority INT NOT NULL __________
    (priority BETWEEN 1 AND 5),                 -- [3]
  status VARCHAR(20) NOT NULL DEFAULT __________ -- [4]
);
```

17.2 Assume the EF Core model is configured and the project can connect to PostgreSQL. Write the TWO dotnet ef commands, in order, to create a migration named AddTickets and then apply pending migrations to the database. Do not write Program.cs configuration. **(2 marks)**

Question 18 Validation filtering and persistence (12 marks)

Complete only the TWELVE numbered blanks [1]-[12]. Each carries 1 mark. Required imports, constructor injection and EF Core mappings already exist. _db.Tickets is a DbSet<Ticket>. TicketSummaryDto has the constructor (long Id, string Title, int Priority). The reporting endpoint permits anonymous submissions for these code exercises; its controller is already implemented.

18.1 Complete the input DTO so Title is required, its maximum length is 120 characters and Priority must be between 1 and 5 inclusive. **(3 marks)**

```
public class CreateTicketDto
{
    [__________]                          // [1]
    [__________]                          // [2]
    public string Title { get; set; } = "";
    public long RoomId { get; set; }
    [__________]                          // [3]
    public int Priority { get; set; }
}
```

18.2 Complete this query to return only Open tickets in the requested room, sorted by highest priority first, projected to TicketSummaryDto and executed asynchronously. Use the method-chain LINQ form shown. **(5 marks)**

```
public async Task<List<TicketSummaryDto>>
    GetOpenTicketsAsync(long roomId)
{
    return await _db.Tickets
        .__________(t => t.RoomId == roomId &&
                         __________)      // [4], [5]
        .__________(t => t.Priority)       // [6]
        .__________(t => new TicketSummaryDto(
            t.Id, t.Title, t.Priority))    // [7]
        .__________();                     // [8]
}
```

18.3 Complete this persistence method. Assume input validation and verification that RoomId exists have already succeeded. The database generates Id. **(4 marks)**

```
public async Task<long> SaveTicketAsync(CreateTicketDto dto)
{
    var ticket = new Ticket
    {
        Title = __________,              // [9]
        RoomId = dto.RoomId,
        Priority = dto.Priority,
        Status = __________              // [10]
    };
    _db.Tickets.__________(ticket);       // [11]
    await _db.__________();               // [12]
    return ticket.Id;
}
```

Question 19 Permission policies and token validation (6 marks)

19.1 Only users whose validated token has the claim perm = tickets.assign may assign a technician. Complete blanks [1]-[2] to configure and apply that policy. Each blank carries 1 mark. The assignment action and its route are already implemented. **(2 marks)**

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AssignTickets", policy =>
        policy.__________("perm", "tickets.assign")); // [1]
});

[Authorize(__________ = "AssignTickets")]             // [2]
public async Task<IActionResult> Assign(long id,
    AssignTicketDto dto) { /* existing implementation */ }
```

19.2 A user has a valid authenticated token and the role Technician, but the token has no tickets.assign permission. State the appropriate HTTP status for the policy failure and explain why the role alone does not satisfy the policy above. **(2 marks)**

19.3 A second token has a valid signature and has not expired, but it was issued by another organization for a different application. Name and explain the TWO validation checks that should prevent the CampusCare API from accepting it. **(2 marks)**

Section C Frontend development with React

Questions 20 to 22 15 marks

CampusCare has a React / Vite dashboard. Use JavaScript and React for the requested code. Questions 21 and 22 are separate implementations of the submission flow. CSS and full project setup are not assessed.

Question 20 Route loading and component identity (4 marks)

20.1 The analytics page is large and most reporters never open it. Name the TWO React features used together to load this page's code on demand and show a fallback while it loads. Explain one performance benefit. **(2 marks)**

20.2 The ticket editor renders rows with key={index}. After a row is deleted or the list is re-sorted, a typed note can appear beside the wrong ticket. Explain why the index is a poor identity here and show the replacement key expression when every ticket has a stable id. **(2 marks)**

Question 21 Submitting a report from React (6 marks)

A public demonstration endpoint POST /api/tickets accepts { title, roomId, priority } and returns the created ticket as JSON with HTTP 201. Required imports, controlled input fields and parent callback onCreated already exist. roomId and priority are numbers. The base URL is import.meta.env.VITE_API_URL.

```
const [title, setTitle] = useState('');
const [saving, setSaving] = useState(false);
const [error, setError] = useState(null);

async function handleSubmit(e) {
    // TODO: complete the handler body for 21.1
}

// Existing form: <form onSubmit={handleSubmit}>...</form>
```

21.1 Write only the handler body. Prevent the default form submission. If title is blank after trimming, set an error and send no request. Otherwise set saving to true, clear the previous error and POST a JSON body to the endpoint. Check the response before using its JSON; call onCreated with the created ticket on success. Store a useful error on failure and set saving to false after either outcome. Do not write the input fields or CSS. **(4 marks)**

21.2 Write only TWO JSX fragments: a Submit button that is disabled while saving and changes its label during submission, and an error message that is rendered only when error is present. **(2 marks)**

Question 22 Cache identity after a mutation (5 marks)

In a separate implementation, TanStack Query manages the room's ticket list. Its query key is ["tickets", roomId]. createTicket is an existing API function. The query client provider and imports are already configured.

22.1 Complete only blanks [1]-[3] so a successful create operation invalidates that room's cached ticket list. Each blank carries 1 mark. **(3 marks)**

```
const queryClient = __________();                  // [1]
const addTicket = __________({                     // [2]
    mutationFn: createTicket,
    onSuccess: () => queryClient.__________({       // [3]
        queryKey: ['tickets', roomId]
    })
});
```

22.2 Explain why roomId belongs in the ticket query key and why invalidation is needed after a successful create operation. Write one sentence for each point. **(2 marks)**

Section D Mobile development with Flutter

Questions 23 to 25 15 marks

CampusCare also has a Flutter reporter app. Use Dart and Flutter. Questions 24 and 25 are separate excerpts; write only the numbered completions. Camera attachments are optional.

Question 23 Mobile layers and a camera feature (5 marks)

23.1 One Flutter screen currently builds widgets, decides whether a report is ready to submit and directly reads/writes ticket data through HTTP and local storage. Assign these THREE responsibilities to suitable layers: UI, service or logic, and repository or data. Write one scenario-specific responsibility for each layer. **(3 marks)**

23.2 A reporter may attach a camera photograph. Before choosing a camera plugin, state ONE compatibility check for a release on both Android and iOS. Then state ONE practical fallback that still lets the reporter submit a maintenance report when camera access is unavailable or denied. **(2 marks)**

Question 24 A validated Flutter form (5 marks)

Complete only blanks [1]-[10]. Each carries 0.5 marks. These excerpts belong to a StatefulWidget State class. The Form is rendered in build(); submit(String title) already exists. The field must reject an empty or whitespace-only title, and the controller must be released when the State is disposed.

```
final _formKey = GlobalKey<__________>();          // [1]
final _titleController = __________();             // [2]

Form(
  key: __________,                                 // [3]
  child: Column(children: [
    TextFormField(
      __________: _titleController,                // [4]
      __________: (value) {                        // [5]
        if (value == null || value.trim().__________) { // [6]
          return 'Enter a title';
        }
        return null;
      },
    ),
    ElevatedButton(
      onPressed: () {
        if (_formKey.__________!.__________()) { // [7], [8]
          submit(_titleController.text.__________()); // [9]
        }
      },
      child: const Text('Report'),
    ),
  ]),
)

@override
void dispose() {
  _titleController.__________();                   // [10]
  super.dispose();
}
```

Question 25 Creating a report through a REST API (5 marks)

Complete only blanks [1]-[10]. Each carries 0.5 marks. ApiService has String baseUrl and an http.Client named _client; http and dart:convert are imported. The demonstration endpoint is public, accepts a JSON ticket and returns {"id": 82, ...} with HTTP 201. Return the created integer id; throw an exception for any other status. Error presentation is handled by the caller.

```
Future<int> createTicket(String title, int roomId) async {
  final response = __________ _client.__________( // [1], [2]
    Uri.__________('$baseUrl/api/tickets'),        // [3]
    headers: {
      '__________': '__________'                   // [4], [5]
    },
    body: __________({                            // [6]
      'title': title,
      'roomId': roomId,
      'priority': 2
    }),
  );
  if (response.__________ != __________) {         // [7], [8]
    throw Exception('Submit failed: ${response.statusCode}');
  }
  final data = __________(response.body)           // [9]
      as Map<String, dynamic>;
  return data['id'] as __________;                 // [10]
}
```

Section E Hosting selection and engineering practice

Questions 26 to 27 10 marks

CampusCare is preparing a release of its web dashboard, API and database. Answer using the given hosting capabilities, integration symptoms and collaboration requirements.

Question 26 Hosting fit and deployment diagnosis (5 marks)

The team has already selected a static host for its React dashboard. Choose the remaining hosting services using ONLY the capabilities given below; no provider prices or current free-tier knowledge are needed.

Service	Given capability
S	Serves built HTML, CSS and JavaScript. Cannot run a persistent .NET process or host PostgreSQL.
A	Runs an ASP.NET Core application or container; provides HTTPS, runtime logs and configuration variables.
D	Runs managed PostgreSQL; provides a database connection endpoint. Does not run application controllers.

26.1 Select a service for the ASP.NET Core API and a service for the database. Justify each choice by naming the runtime or persistence requirement it meets. **(2 marks)**

26.2 Give ONE targeted fix for EACH independent deployment problem: (i) the API works in a mobile client, but the dashboard browser reports that its origin is not allowed; (ii) runtime logs say the deployed PostgreSQL database has no Tickets table because no migrations were applied; (iii) the HTTPS dashboard attempts to call an HTTP API URL and the browser reports mixed content. Write three short points. **(3 marks)**

Question 27 Resolving a conflict and controlling a release (5 marks)

A merge conflict in src/formatTicketTitle.js contains the following lines. The agreed behaviour is to remove leading/trailing spaces AND convert the title to uppercase.

```
<<<<<<< HEAD
return title.trim();
=======
return title.toUpperCase();
>>>>>>> feature/title-display
```

27.1 Write the single return statement that preserves both required behaviours. Do not retain any conflict markers. **(1 marks)**

27.2 Assume you have saved and tested the corrected file. Write the TWO Git commands to stage this file and complete the merge with a suitable commit message. **(2 marks)**

27.3 The PR checks show: frontend build passed, lint passed, API tests failed. Explain the appropriate release decision. Then name the delivery approach in which a successful build is prepared automatically but production release still needs a human decision. **(2 marks)**

Section F Agentic software development

Questions 28 to 30 15 marks

CampusGuide supports staff by consulting campus policies and ticket data. Explain agent concepts in plain language. No Python implementation is required. The source passage and tool results in the questions are practice scenario data.

Question 28 Reading an agent tool trace (5 marks)

CampusGuide is an AI assistant for maintenance staff. A user asks, "How many Open tickets in room R101 have priority at least 4?" The trace below contains the complete tool results. No private model reasoning is shown or required.

Step	Recorded event
1	The model selects list_open_tickets(room="R101").
2	Tool result: [{"id":82,"priority":5}, {"id":89,"priority":2}]. This is the complete Open-ticket list for R101.
3	The model selects count_high_priority with those two tickets and minimum_priority=4.
4	Tool result: {"count":1}.

28.1 Using this trace, explain the role of reasoning or decision-making, action, and observation in a ReAct-style agent. Give one concrete example from the trace for each. **(3 marks)**

28.2 Write a one-sentence final answer supported by the results. Keep the answer's scope consistent with the user's question. **(1 marks)**

28.3 If step 4 instead reports a timeout with no count, explain what the assistant should do before claiming a count. Give one appropriate response or recovery behaviour. **(1 marks)**

Question 29 Grounded answers and retrieval evaluation (5 marks)

29.1 The user asks, "There is a burning smell from a socket. What should I do, and how many minutes will the repair take?" The only retrieved passage is labelled [Safety Guide page 4] and states: "If an electrical hazard is suspected, keep clear of the affected area and telephone campus security on extension 202." It gives no repair-time estimate. Write a TWO-sentence answer that cites the supported safety advice and handles the unsupported repair-time request appropriately. **(2 marks)**

29.2 For a different test query there is exactly ONE known relevant policy passage. It appears at rank 8, while the retriever returns only the top 3 passages. (i) Identify whether this failure starts in retrieval or answer generation. (ii) Calculate Recall@3, showing the relevant retrieved count and total relevant count. (iii) Propose ONE targeted retrieval change that can include the known passage in the model's context. You do not need to compute embeddings or RRF. **(3 marks)**

Question 30 Agent handoffs and recoverable approval (5 marks)

CampusGuide uses a researcher specialist and a planning specialist. The researcher saves its findings as evidence/R101_report.md for ticket 82. The planner must propose repair steps with source references. A separate workflow proposes shutting down an affected circuit; campus policy requires an authorized staff member's approval before that action executes.

30.1 Write a small example structured handoff from the researcher to the planner. Include these FOUR fields: task, ticket_id, evidence_refs and expected_output. JSON or clearly labelled lines are accepted. Each useful field carries 0.5 marks. **(2 marks)**

30.2 Give ONE reason to pass the saved evidence reference instead of repeatedly copying the entire conversation and report into every specialist prompt. **(1 marks)**

30.3 The approval workflow is paused, but its state is held only in an in-memory checkpointer. Explain how to preserve and resume the pending workflow after a server restart. Address both storage of the state and resumption after an explicit approval or denial. **(2 marks)**

End of model paper